American International University-Bangladesh (AIUB)

Department of Computer Science Faculty of Science &Technology (FST)

Spring 24 /25

Software Quality Assurance and Testing Test

Web-Based Banking System

Section: B

A Report submitted By

SN	Student Name	Student ID
1	S.M.Sayed Arifin Omi	22-46613-1

Under the Supervision

Abhijit Bhowmik
Associate Professor
Department of Faculty of Science & Technology
American International University-Bangladesh

Software Test Plan for
Banking System

Version 1.0

Prepared by

S.M. Sayed Arifin Omi

American International University-Bangladesh

26 June,2025

Table of Contents

1. TEST PLAN IDENTIFIER: RS-MTP01 3

2. REFERENCES 3

3. INTRODUCTION 4

1. 3.1 Background to the Problem. 4
2. 3.2 Solution to the Problem 4

4. REQUIREMENT SPECIFICATION 6

1. System Features. 6
2. System Quality Attributes 8
3. System Interface 10
4. Project Requirements 15

5. FEATURES NOT TO BE TESTED 16

6. TESTING APPROACH 17

1. Testing Levels 17
2. Test Tools 18
3. Meetings 20

7. TEST CASES/TEST ITEMS 21

1. Add customer 21
2. Fund Transfer 22
3. Add Marchant 23
4. Set Appointment 24
5. Loan Application 25
6. Pay Bill 26

8. ITEM PASS/FAIL CRITERIA 26

9. TEST DELIVERABLES 27

10. STAFFING AND TRAINING NEEDS 27

11. RESPONSIBILITIES 28

12. TESTING SCHEDULE 30

13. PLANNING RISKS AND CONTINGENCIES 30

TEST PLAN IDENTIFIER: Web based Banking Management System

REFERENCES

o Software Requirement Specification (SRS) Document

o https://www.ucb.com.bd

1.Introduction

Background to the Problem

In modern banking, customers demand 24/7 access to their accounts, especially for transactions like fund transfers, bill payments, and loan applications. However, traditional banking systems often rely heavily on physical interactions, and some existing online systems are slow, unintuitive, or insecure. These shortcomings lead to poor user experiences, particularly for customers who are accustomed to smooth digital services in other domains.

Root Cause:

The lack of a streamlined, secure, and responsive web-based platform leads to inefficient transactions, delays in loan approvals, and unsatisfactory customer service. Furthermore, systems that don't scale properly during peak usage hours can experience performance degradation and login/session issues.

Solutions to the Problem: (Proposed Solutions)

Real-Time Fund Transfer Engine: Replaces batch processing with direct transaction handling using web APIs to support instant transfers.

Responsive Web Interface: A clean, browser-based interface that adjusts to different screen sizes (desktop, tablet, etc.) to serve both customers and bank staff.

Secure Admin Control Panel: Allows bank employees to manage accounts, review transactions, and approve loans securely and efficiently.

Why is this solution appropriate:

This approach ensures improved user experience, faster transaction handling, and reduced dependency on physical branches, meeting the demands of both personal and business customers.

Feasibility:

The project is technically and financially feasible. With common tech stacks like PHP/MySQL or Python/Django, and integration with payment gateways or REST APIs, development is straightforward. Most browsers support the platform, and no device-specific development is required.

Short Description of the Software

The iBanking System is a browser-based online banking platform designed to offer secure and convenient services to customers remotely.It includes features like login/authentication, account registration, fund transfers (internal and external), utility bill payment, loan application, and account management.The system supports role-based access for admins and customers and includes real-time transaction validation and digital receipts. It's designed for high availability and performance even during peak banking hour

Purpose:

The purpose of the **iBanking System** is to bring full-fledged banking capabilities to users through a modern, accessible, and secure online interface.By shifting banking operations online, it minimizes physical visits to bank branches, streamlines operations for the bank, and improves customer satisfaction by offering digital convenience for everyday transactions and account management.

Existing Studies in the Problem Area

Various banks have already implemented secure, scalable web-based banking portals that have significantly improved operational efficiency:

Dutch Bangla Bank Web Portal: Allows customers to perform all transactions online, including fund transfers and mobile wallet top-ups.

Standard Chartered Online Banking: Offers bill payment, investment, and loan tracking features through a web dashboard.

Sonali Bank's Web Banking (Bangladesh): A government-supported online system offering fund transfers and transaction summaries to rural and urban users alike.

International Examples (Wells Fargo, HSBC): Global banks offer comprehensive, role-based online banking solutions through web browsers, supporting both personal and business accounts.

These implementations confirm that secure, user-friendly iBanking portals are essential in modern financial systems.

4. REQUIREMNT SPECIFICATION

4.1 System Features:

1. System Login

Description:
Enables users (customers/admins) to securely log in using a username and password.

Functional Requirements:

- Users must log in using valid credentials.
- After 3 failed attempts, a CAPTCHA or verification code appears.

- After 5 failed attempts, the account is locked for 1 hour.
- Session timeout after 10 minutes of inactivity.

Priority Level: High
Precondition: User must be registered in the system.

1. **User Registration**

Description:
Allows new users to create an account with required personal and contact information.

Functional Requirements:

- System must validate email and phone number uniqueness.
- Passwords must meet minimum strength criteria.
- A confirmation email is sent upon successful registration.

Priority Level: High

Precondition: The user is not already registered.

2. Fund Transfer

Description:
Enables users to transfer funds between internal accounts or to other users' accounts.

Functional Requirements:

- Transfers allowed to own accounts or to other customers of the same bank.
- For external transfers, IBAN or external account number must be validated.
- Transfers after banking hours are scheduled for the next business day.
- Confirmation and receipt generated after successful transfer.

Priority Level: High
Precondition: User is logged in and has a valid account with sufficient balance.

3. Transaction History

Description:
Allows users to view and filter past transactions.

Functional Requirements:

- Filter by date range, transaction type (credit/debit), and amount.
- Users can download or email reports in PDF or CSV format.

Priority Level: Medium

Precondition: User is logged in.

4. Apply for Loan

Description:
Lets users submit loan applications online.

Functional Requirements:

- Form must include loan purpose, amount, duration, and income details.
- Admins receive application for review and approval.
- Users receive notifications about loan status (Pending, Approved, Rejected).

Priority Level: High
Precondition: User must be logged in and have completed KYC.

5. Utility Bill Payment

Description: Users can pay electricity, water, or internet bills online.

Functional Requirements:

- User selects bill type and provider.
- Inputs account number and amount.
- After confirmation, payment is processed and receipt generated.

Priority Level: Medium
Precondition: User must be logged in with a valid bank balance.

6. Admin Dashboard

Description:
Admin panel for monitoring user activity, managing accounts, and approving loans.

Functional Requirements:

- Admins can view transaction logs, loan applications, and registered users.
- Admins can activate/deactivate accounts or reset passwords.
- Role-based access control ensures only admins can see this panel.

Priority Level: High

Precondition: Admin is logged in.

7. Profile & Security Settings

Description: Users can update their profile, change password, and enable 2FA.

Functional Requirements:

- Passwords must be changed with email verification.
- 2FA via OTP to registered mobile or email.
- Email and mobile updates require verification.

Priority Level: Medium

Precondition: User is logged in.

4.2 System Quality Attributes

4.2.1 Performance

The iBanking System is designed to deliver fast response times and support high transaction throughput, even during peak hours.

- Response Time: All user actions (e.g., login, transfers) should receive system feedback within 2 seconds under normal usage.
- Throughput: The system must handle at least 100 concurrent transactions per second without performance degradation.
- Scalability: The architecture should support vertical and horizontal scaling to accommodate future growth in users and data volume.

4.2.2 Reliability

Reliability ensures uninterrupted access and trustworthy operations, especially for financial transactions.

- Uptime: The system should maintain 99% or higher availability throughout the year.
- Transaction Accuracy: All transactions must complete without duplication or loss of data, with automated rollback on failure.
- Failure Recovery: System should detect faults automatically and recover from interruptions with minimal manual intervention.
- Data Consistency: Balances and records must remain synchronized across all modules after each transaction.

4.2.3 Security

Security is a core requirement due to the sensitive nature of financial and personal data.

- Authentication & Authorization: Only verified users may perform sensitive tasks, based on assigned roles (admin/customer).
- Data Encryption: All data in transit and at rest should be protected using modern cryptographic standards (e.g., AES-256, HTTPS).
- Audit Trails: Every login, transaction, and data modification must be logged for monitoring and forensic analysis.

- Two-Factor Authentication (2FA): Added security for logins and high-value transactions via OTP to registered email or phone.

- Threat Protection: System must be hardened against threats like SQL injection, CSRF, XSS, and DDoS attacks.

4.2.4 Usability

The system must be intuitive for users of all skill levels, ensuring a smooth digital banking experience.

- User-Friendly Design: Interfaces should be clear, responsive, and easy to navigate across devices.

- Accessibility Compliance: Follows WCAG standards to support users with disabilities.

- Error Feedback: Users should receive informative error messages to guide them through resolving issues.

4.3 System Interface

UI Design :

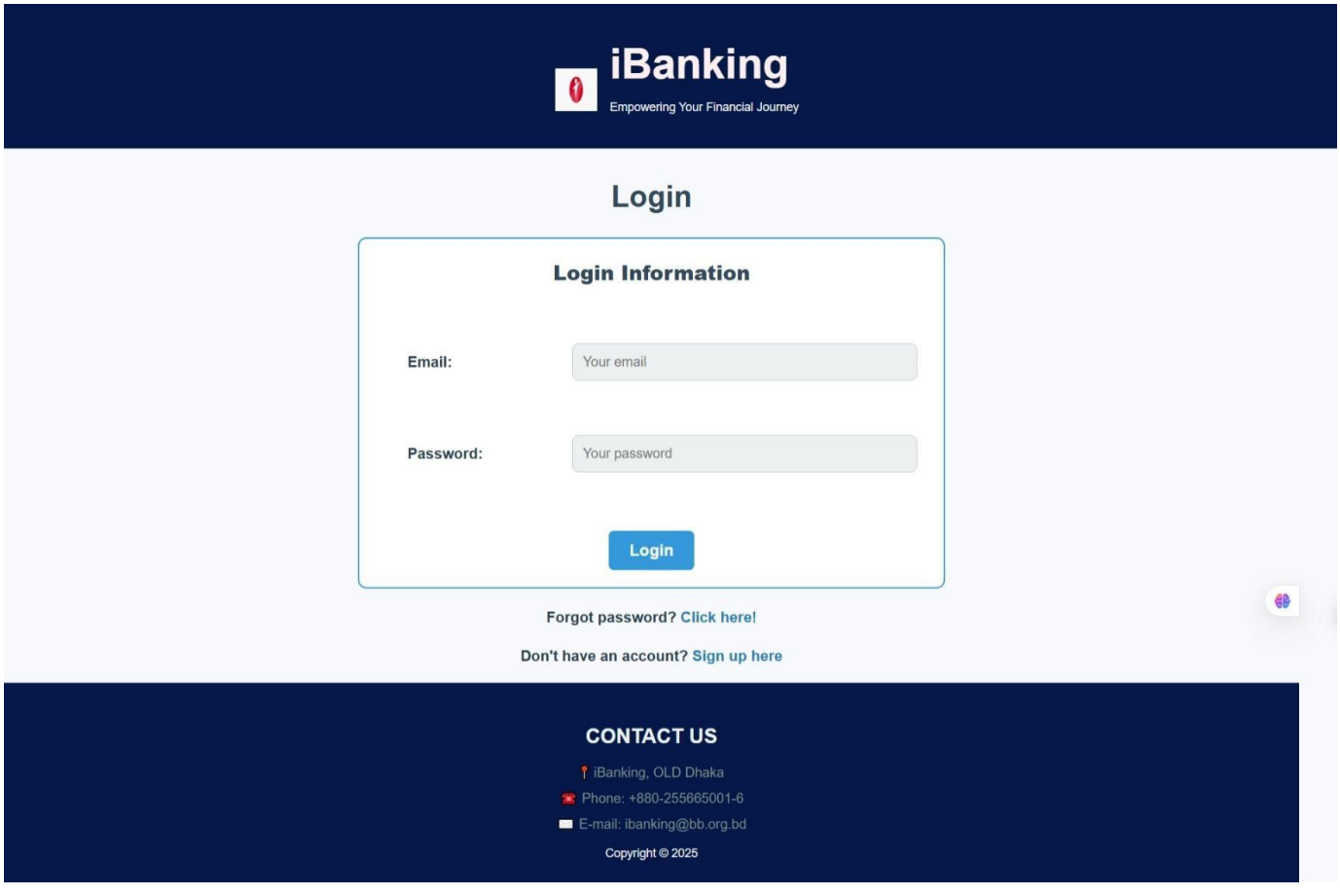


Fig1: Sign In Page

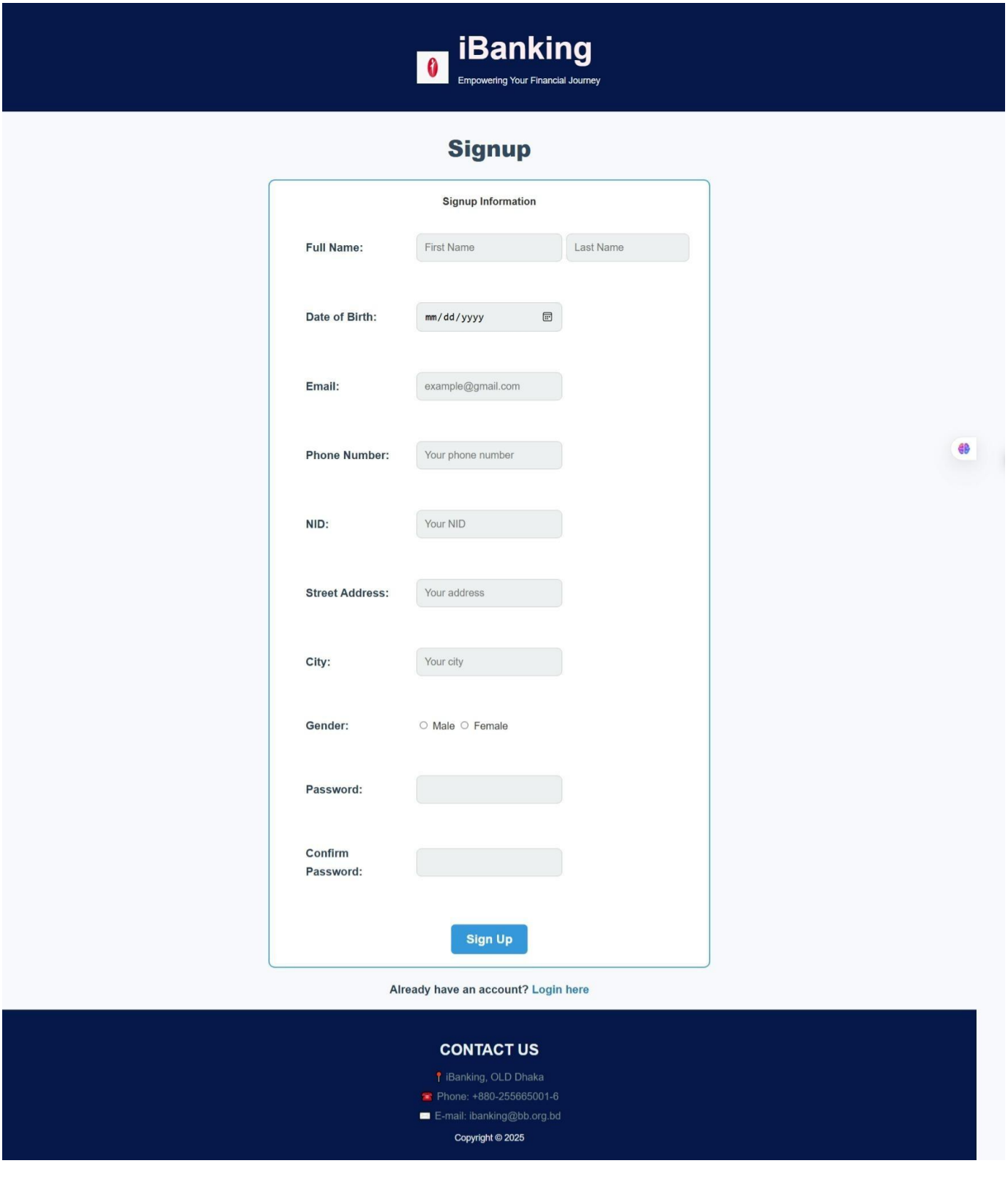


Fig2: Sign Up Page

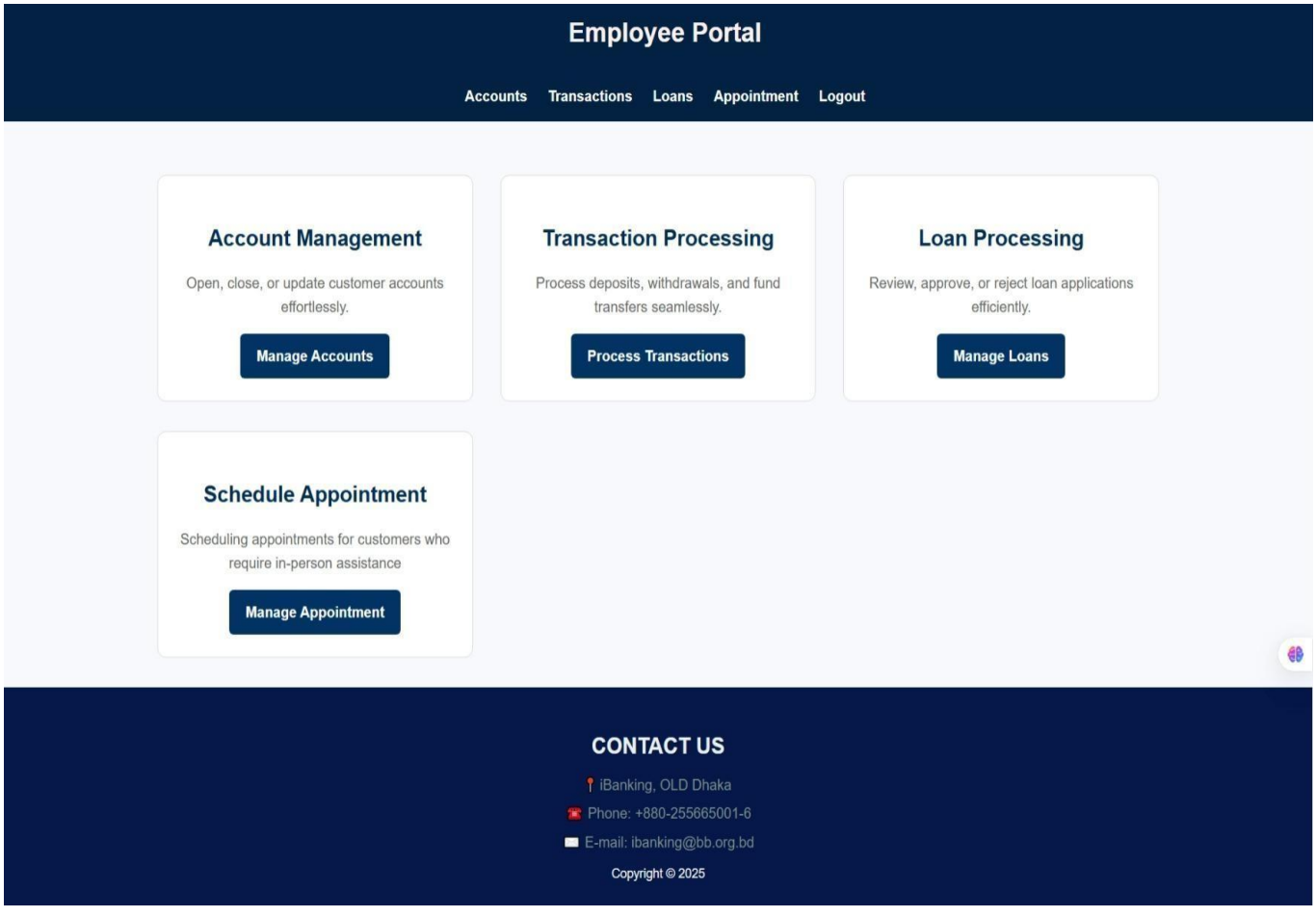


Fig3: User DashBoard

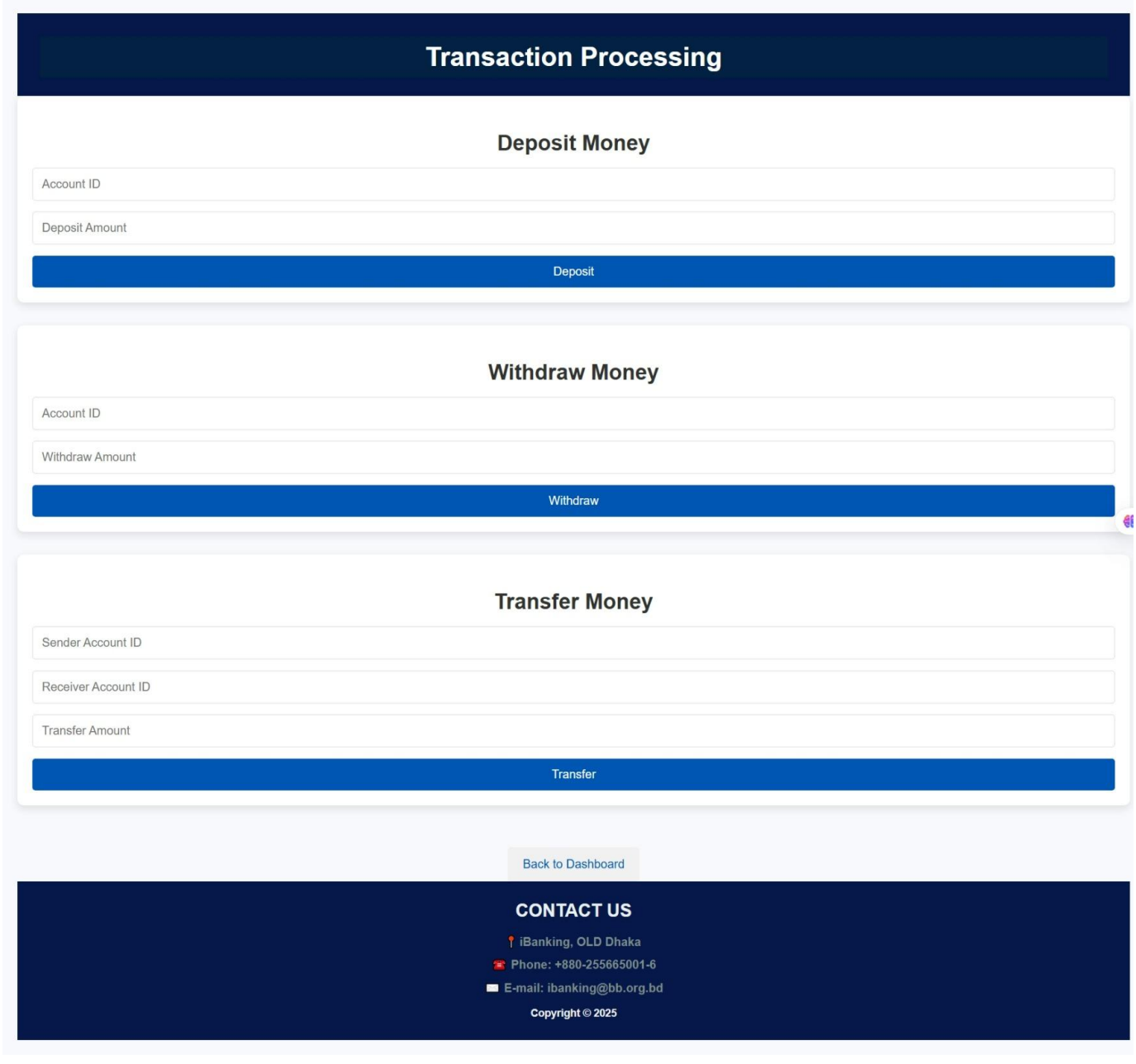


Fig4: Transaction Processing

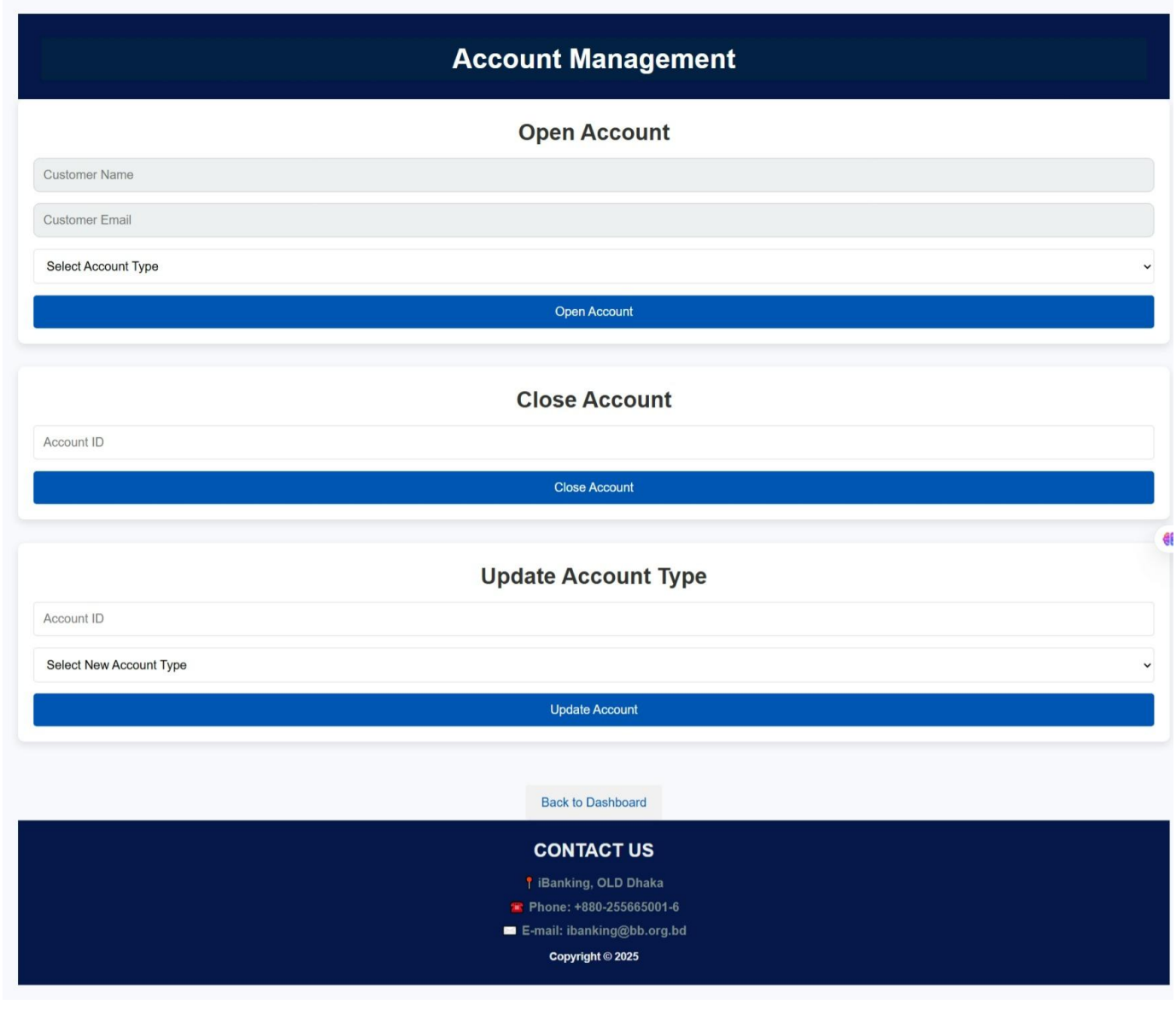


Fig5: Account Management

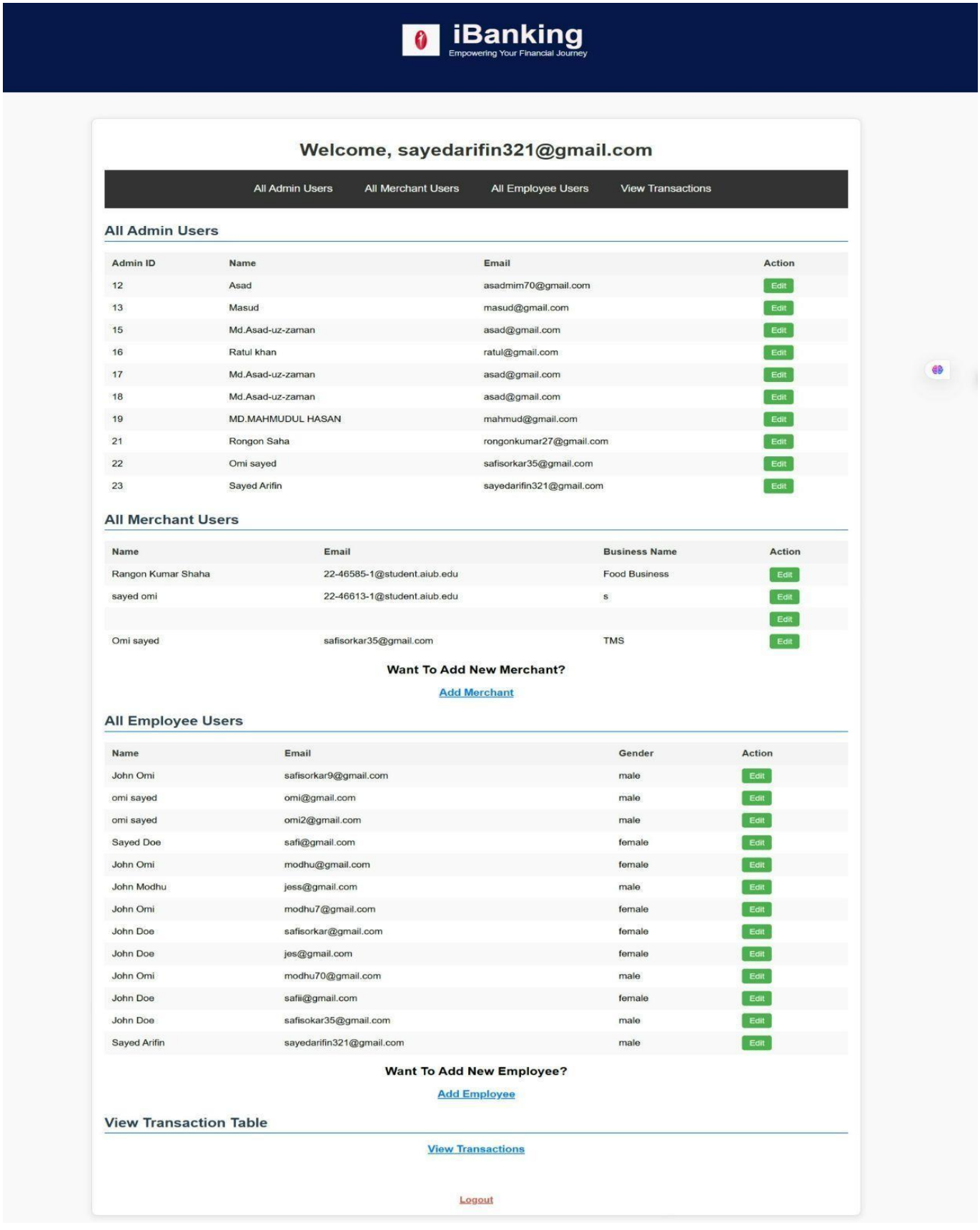


Fig6: Admin Dashboard

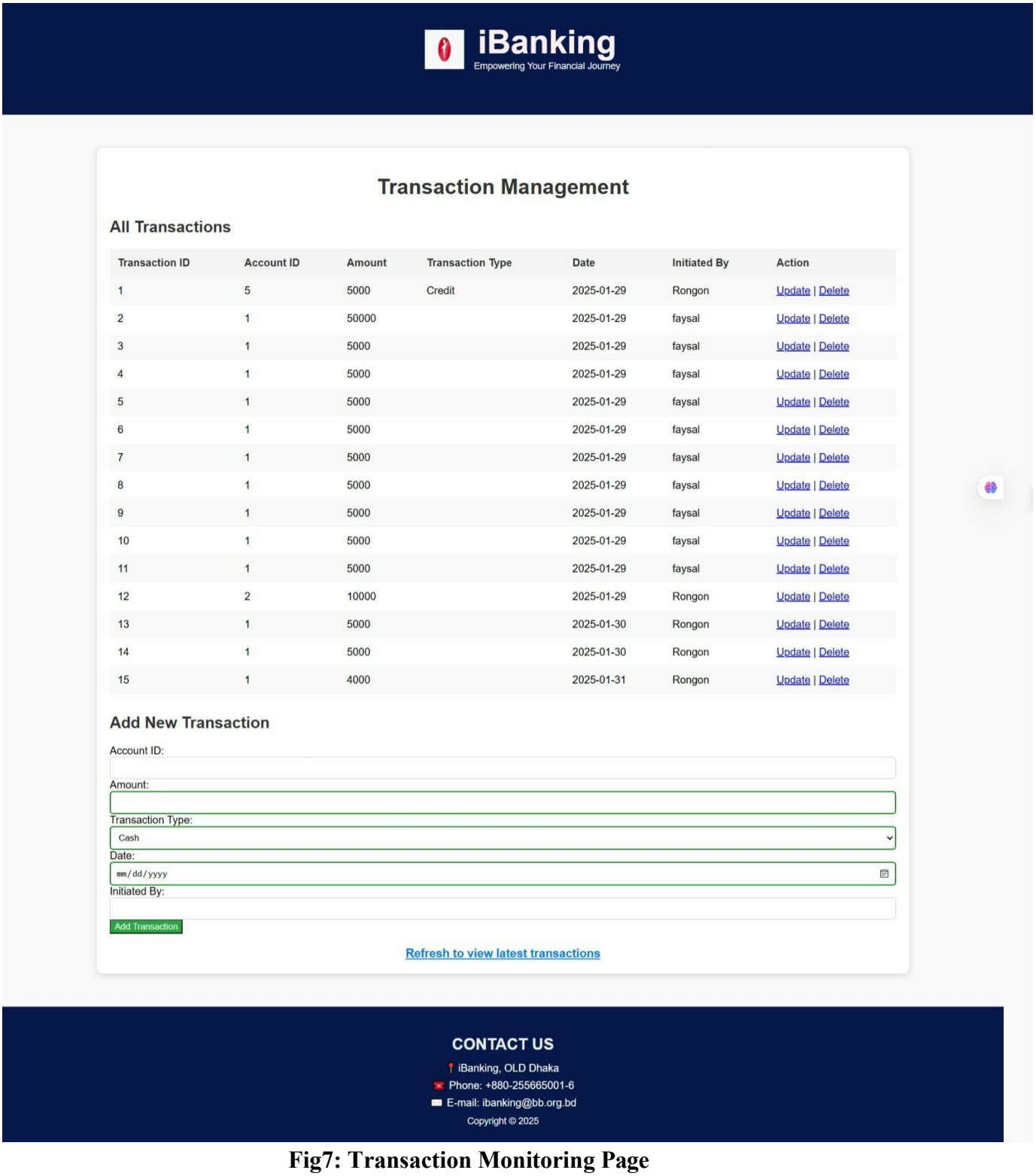


Fig7: Transaction Monitoring Page

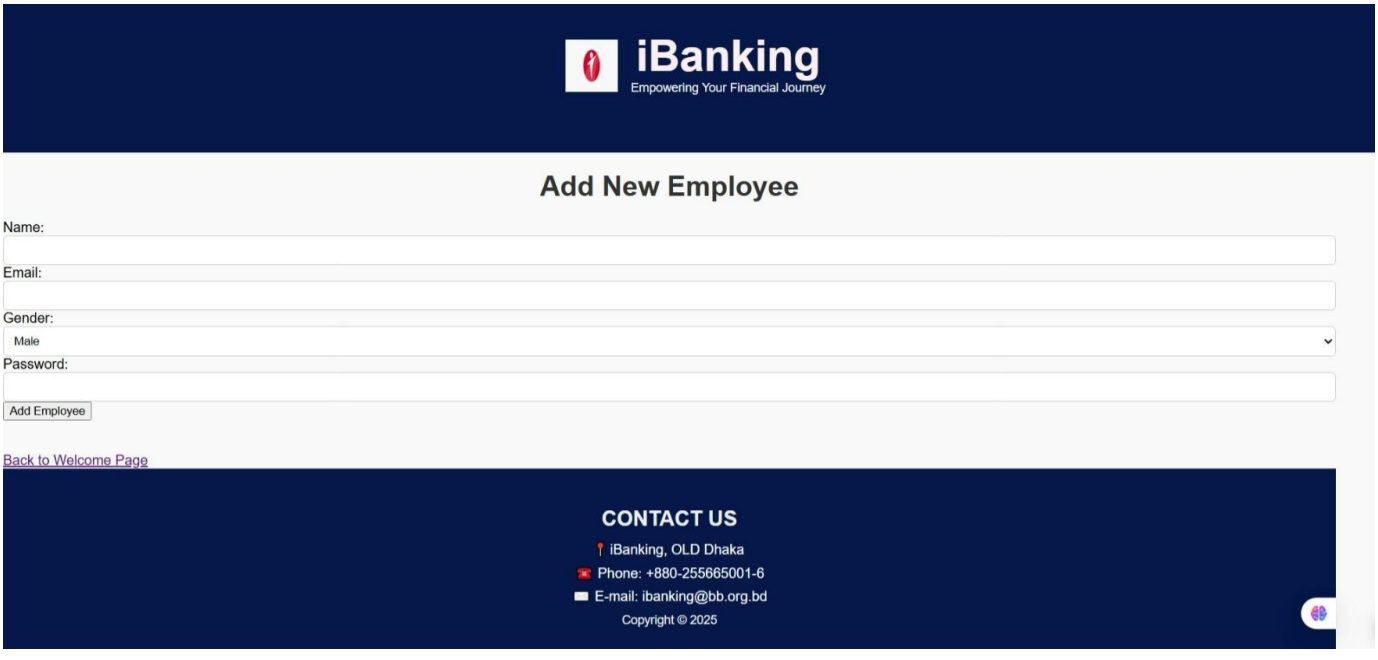


Fig8: ADD New Employee

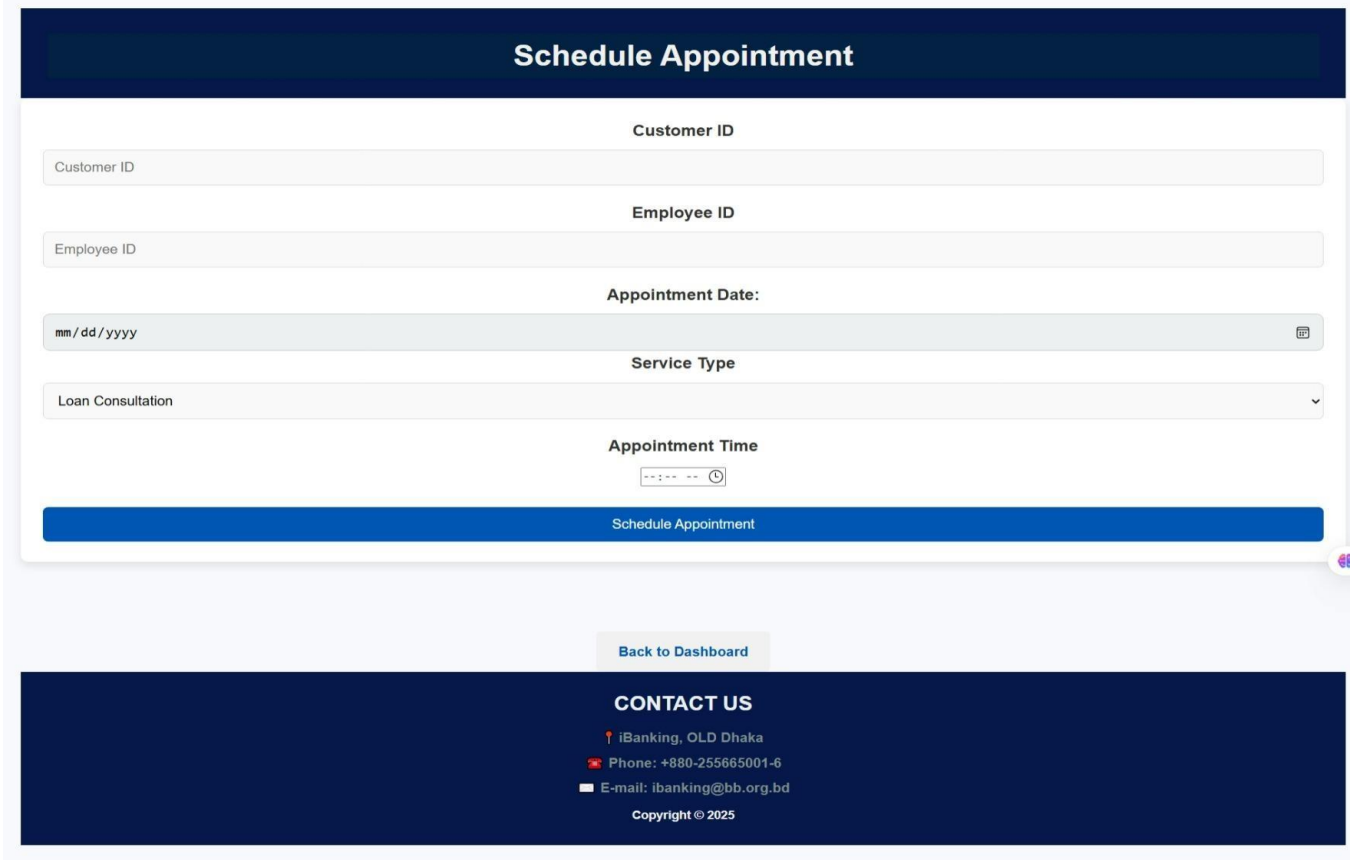


Fig9: Schedule Appointment

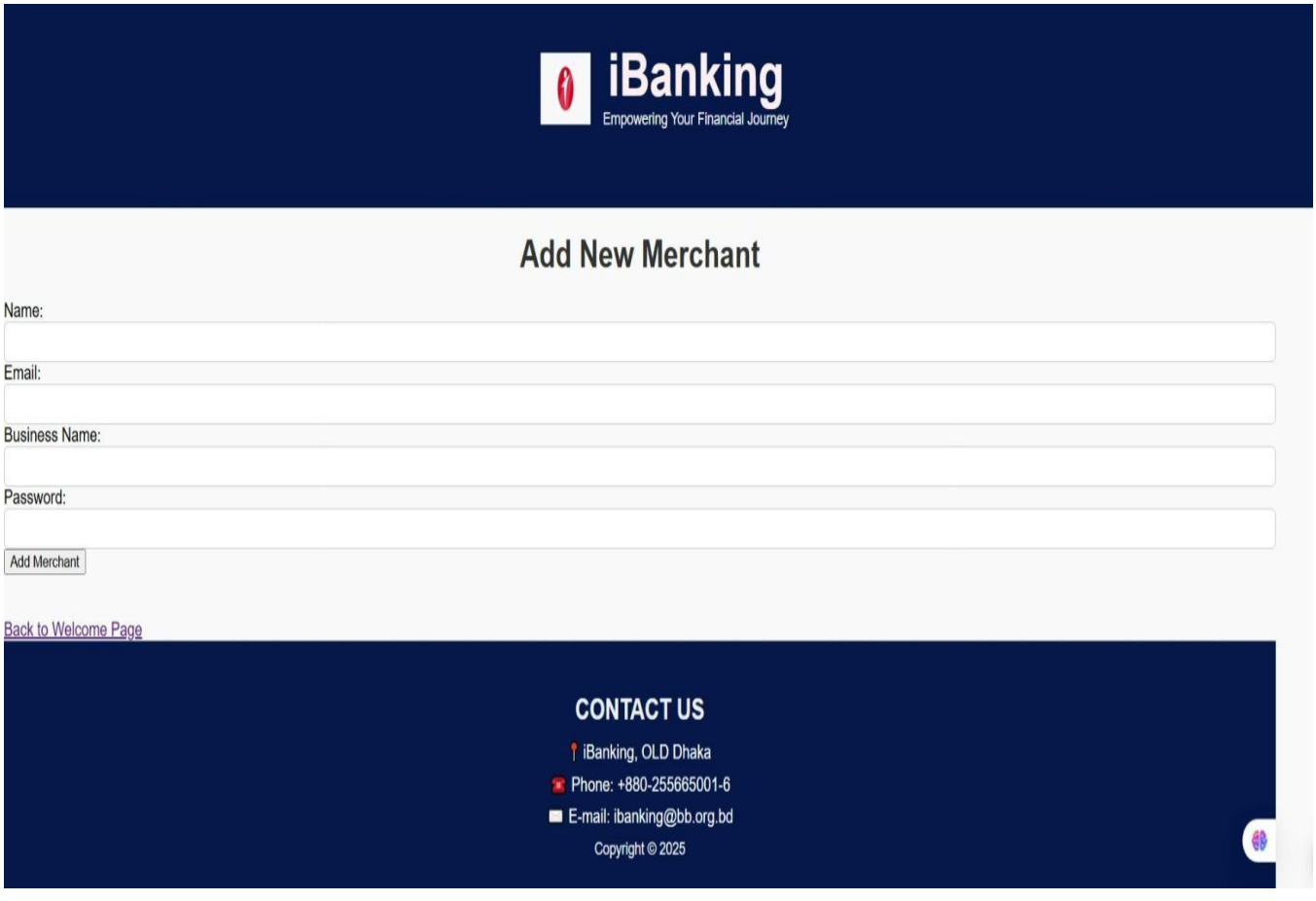


Fig10: ADD New Merchant

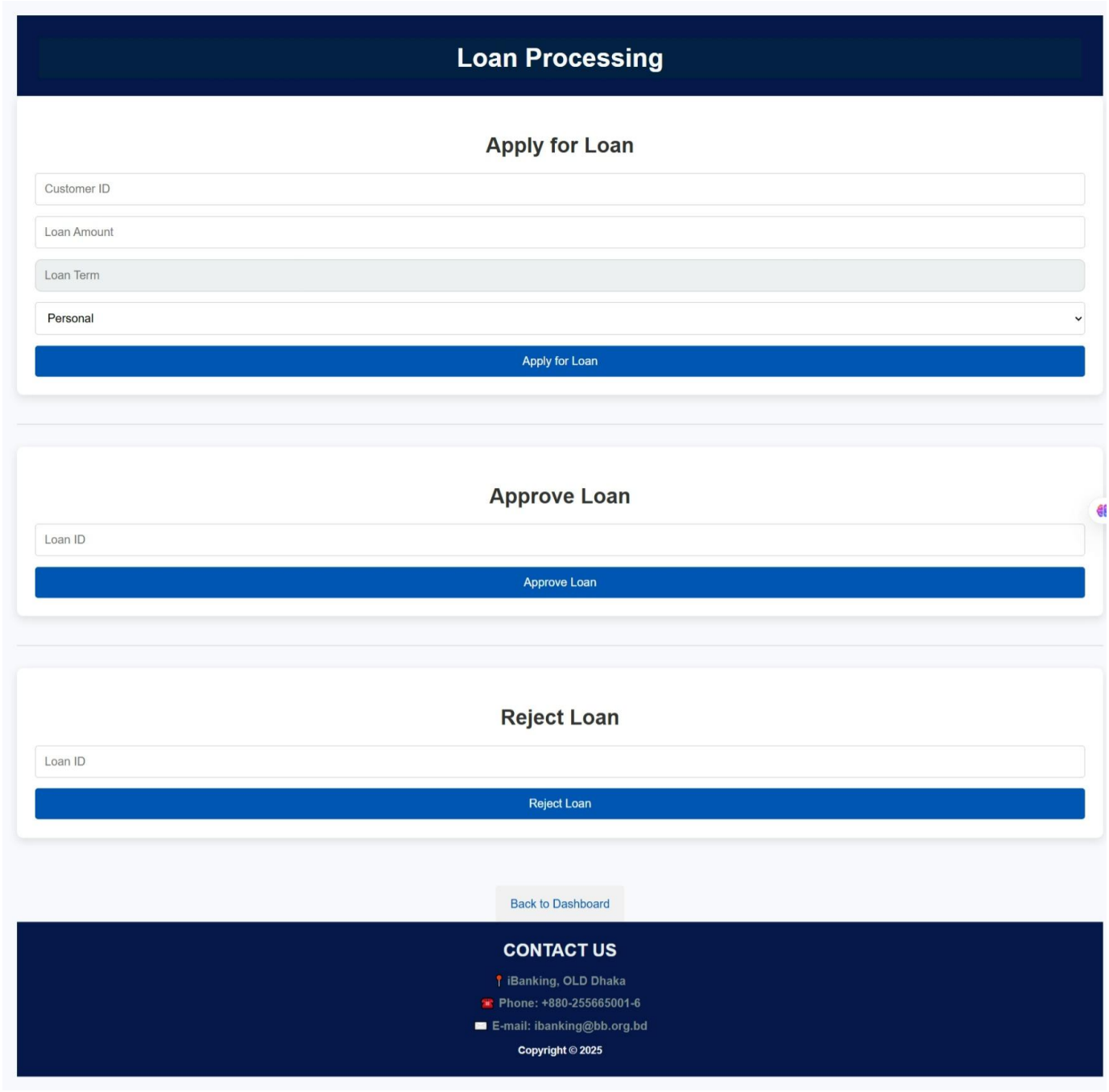


Fig11: Loan Processing

4.4 Project Requirements

The iBanking System development effort has been estimated using the **COCOMO (Constructive Cost Model)**, which is widely used for software cost estimation. Given the scale and complexity of the system, we are treating it as a **semi-detached project** involving both routine development and some embedded financial operations logic.

- **Estimated Codebase**

  - **Estimated LOC (Lines of Code):** 20,000

- **Effort Estimation**

Using the COCOMO formula:
Effort (PM) = 3.6 × (KLOC)^1.20

PM = 3.6 × (20)^1.20 ≈ **135.7 person-months**

- **Time Estimation**

Development Time (TDEV) = 2.5 × (PM)^0.32

TDEV = 2.5 × (135.7)^0.32 ≈ **11.3 months**

- **Team Size**

Average Team Size (People) = PM / TDEV

135.7 / 11.3 ≈ **12 developers/testers**

Budget Estimation

Item	Amounts
Developer Salaries (12 × 50,000 BDT × 11.3 months)	6,78,000 BDT
Hosting, Cloud Infrastructure	50,000 BDT
Tools & Software Licenses	25,000 BDT
Miscellaneous & Contingency	27,000 BDT
Total Estimated Budget	780,000 BDT

- **Development Duration:** Approx. 11–12 months
- **Team Composition:** Developers, UI/UX designer, QA testers, and project manager
- **Tools Involved:** IDEs, Selenium, SQL consoles, API testing tools, version control (e.g., Git)

This budget and timeline provide a realistic foundation for a moderately scaled, secure, and responsive web-based banking system tailored to customer and administrative needs.

5. FEATURES NOT TO BE TESTED

Some components of the system will not be included in the current testing cycle due to their dependency on third-party services or because they are outside the scope of this phase.

Excluded Features:

- Third-Party Payment Gateways:
  Integration with external gateways (e.g., SSLCommerz, bKash, VisaNet) will not be tested, as these services have their own certified testing processes. We will only verify API call triggers and response handling.

- Personalized Recommendations:
  Advanced features like user-specific financial recommendations or AI-based analytics (e.g., loan suggestions, spending tips) are deferred for a later phase.

- Mobile App Functionality:
  This version of the test plan focuses only on the web-based system. Mobile responsiveness will be manually reviewed, but native mobile app features (if any) are not included.

6. TESTING APPROACH

6.1. Testing Levels

The testing for the Real-time Funds Transfer Banking System project will consist of Unit, Integration, System, and Acceptance test levels. Given the critical nature of the banking system and the need for high reliability and security, we will employ a comprehensive testing strategy involving both the development team and dedicated testing personnel.

Unit Testing: Unit testing will be conducted by individual developers for each module. The primary focus will be to validate the functionality of individual units or components, such as user login, registration, transaction processing, and loan management modules. Each developer will test their assigned components and provide proof of unit testing to the development team leader. Unit tests will include validating specific inputs, outputs, and edge cases for each module.

System/Integration Testing: System and Integration Testing will focus on how different components and features of the system work together as an integrated whole. This level will also test interactions between the banking system and external components, such as third-party payment gateways and SMS/email notifications. The test manager will work with the development team to test integrated modules (e.g., loan processing, transaction monitoring, account management). Testing will verify the seamless flow of data between different modules and validate cross functional requirements, such as security and data consistency.

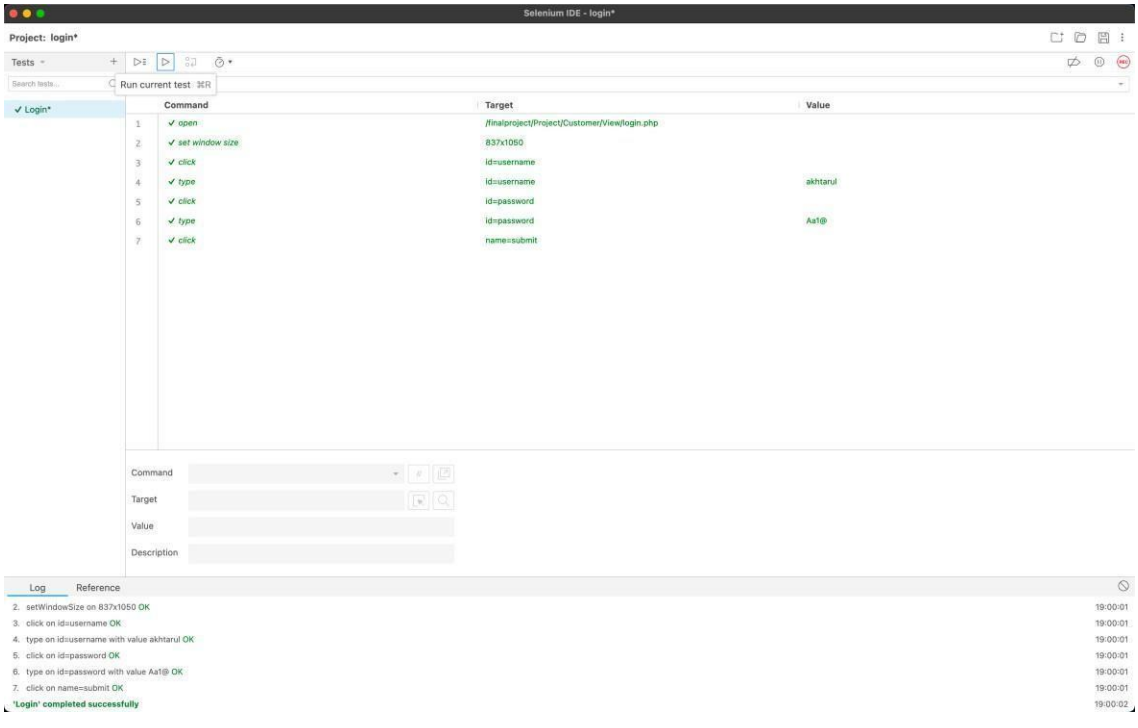
User Acceptance Testing (UAT): Acceptance testing will be conducted by end users (Admin, Employee, Merchant, Customer) in the system's production environment. The goal of UAT is to ensure that the system meets all the business requirements and that it is user-friendly and operationally efficient. End users will test the system with the assistance of the test manager and development team. Each user role will be tasked with executing real-world scenarios, such as account management for employees, payment processing for merchants, and bill payments for customers.

This testing approach will ensure that the system is thoroughly tested at all levels, from individual modules to fully integrated components, and that it meets the needs of all users before final deployment.

6.2. TEST TOOLS

For the project required testing tools we are using Selenium ID

SELENIUM IDE: Selenium IDE is an automation Testing tools. Selenium automates browser-based web applications. Firstly, open Selenium Ide from browser extension. Then open our project and copy the project URL provide it in selenium playback base URL. Then start recording our full project testing. After that stop recording and click RUN ALL TEST then selenium automation testing started and detect the bugs of this project.



c

Fig.1- User Login

For testing, we must include the User login page's URL link. Then select the "record" option, and a new window will appear. Then we have to close the window. Once more, the recorder must be stopped before the current test may be run. Here, the testing of our User Login page has been successful.

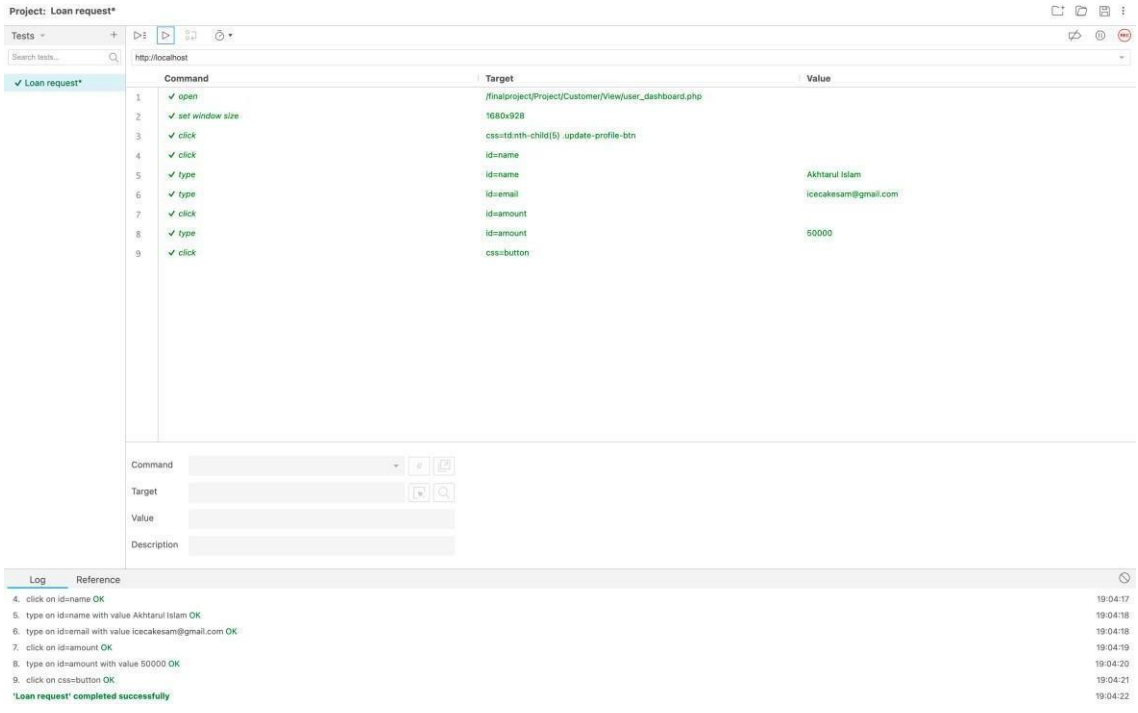


Fig.2- Loan Request

For testing, we must include the Loan Request page's URL link. Then select the "record" option, and a new window will appear. Then we must close the window. Once more, the recorder must be stopped before the current test may be run. Here, the testing of our Loan Request page has been successful.

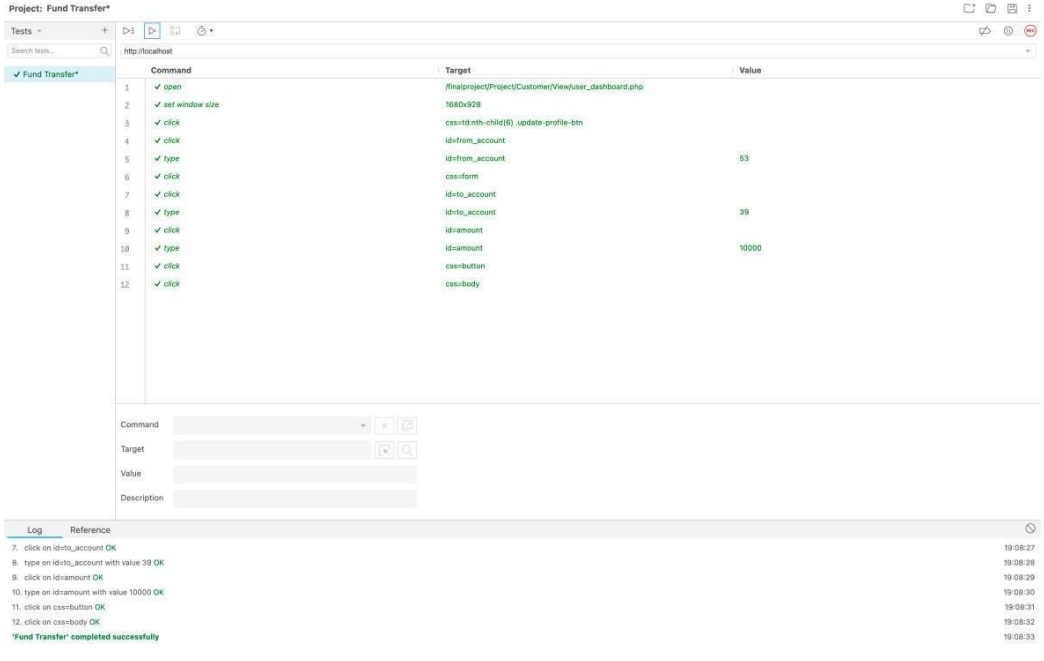


Fig.2- Fund Transfer

For testing, we must include the Fund Transfer page's URL link. Then select the "record" option, and a new window will appear. Then we have to close the window. Once more,

the recorder must be stopped before the current test may be run. Here, the testing of our Fund Transfer page has been successful.

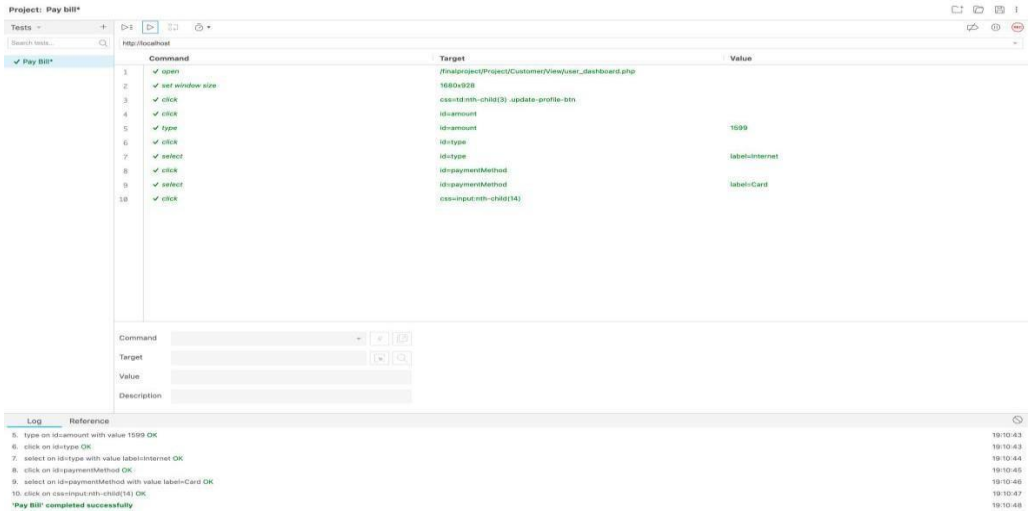


Fig.4- Pay Utility Bill

For testing, we must include the Pay Utility Bill page's URL link. Then select the "record" option, and a new window will appear. Then we have to close the window. Once more, the recorder must be stopped before the current test may be run. Here, the testing of our Pay Utility Bill page has been successful.

6.3 Meetings

Distributing the proper task to the right individual is one of the most important parts of becoming a successful testing team. So, in order to be a successful team, it is critical to hold regular team meetings. As a result, the test team will meet once a week to review each member's progress and determine whether or not they have completed their allocated tasks. And see if they're experiencing any issues with testing; if they are, the entire team will help him and attempt to resolve the issue as soon as possible. Once every two weeks, the testing team lead will meet with the development and project manager. If there is an emergency, an urgent meeting will be scheduled.

TEST CASES/TEST ITEMS

Test Case 1:

Project Name: Web-based Banking System			Test Designed by: Omi	
Test Case ID: WBS_1			Test Designed date: 01/06/25	
Test Priority: High			Test Executed by: Omi	
Module Name: User Registration			Test Execution date: 04/06/25	
Test Title: Register iBanking User				
Description: Test user registration functionality				
Precondition: User must not have an existing account with the same email or username				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to the Registration page 2. Enter Name 3. EnterEmail 4. Enter Phone 5. Enter Password 6. Click Register	Name: Fahim Reza Email: fahim.reza01@gmail.com Phone: +8801785432123 Password: Fahim@123	Confirmation email sent; user added to DB	As expected	Pass
Post Condition: New user is stored in the database and receives a registration confirmation via email.				

Test case 2:

Project Name: Web-based Banking System			Test Designed by: Omi	
Test Case ID: WBS_2			Test Designed date: 1/06/25	
Test Priority: High			Test Executed by: Sayed Omi	
Module Name: Fund Transfer			Test Execution date: 03/06/25	
Test Title: Fund Transfer as Customer				
Description: Test fund transfer page				
Precondition: User must be logged in as Customer				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to the Funds Transfer page 2. Enter From ID 3. Enter To Account 4. Select Amount 5. Click Transfer Funds	From ID: 39 To Account: 11084616523 Amount: 30000	Fund Transfer is successful	As expected	Pass
Post Condition: User is validated with database as Customer and successfully logged in to account. The account session details are logged in the database.				

Test case 3:

Project Name: Web-based Banking System			Test Designed by: Omi	
Test Case ID: WBS_3			Test Designed date: 2/06/25	
Test Priority: Medium			Test Executed by: Omi	
Module Name: Pay Bills			Test Execution date: 10/06/25	
Test Title: Pay Utility Bill				
Description: Test bill payment module				
Precondition: User must be logged in with a valid balance				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to Pay Bill page 2. Select bill type 3. Enter amount 4. Select payment method 5. Confirm payment	Type: Electricity Amount: 1500 Payment Method: Bank Account	Bill paid and receipt generated	As expected	Pass
Post Condition: Transaction is logged, and receipt is available in user's dashboard.				

Test Case 4:

Project Name: Web-based Banking System			Test Designed by: Sayed Omi	
Test Case ID: WBS_4			Test Designed date: 01/06/25	
Test Priority: High			Test Executed by: Omi	
Module Name: System Login			Test Execution date: 20/06/24	
Test Title: User Login with Valid and Invalid Credentials				
Description: Test secure login mechanism with success and failure scenarios				
Precondition: User must have a registered account with valid credentials				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to the Login page 2. Enter valid username and password 3. Enter wrong password 3 times 4. Enter wrong password 5 times	Username: fahimUser1 Password: Fahim@123 Wrong Password: Test@123 Wrong Password: Wrong#456	Account locked for 1 hour	As expected	Pass
Post Condition: System tracks login attempts and enforces security policies.				

Test Case 5:

Project Name: Web-based Banking System			Test Designed by: Sayed Omi	
Test Case ID: WBS_5			Test Designed date: 02/06/25	
Test Priority: Medium			Test Executed by: Omi	
Module Name: Transaction History			Test Execution date: 06/06/25	
Test Title: View and Export Transaction History				
Description: Test user's ability to view, filter, and download past transactions				
Precondition: User must be logged in and have completed transactions				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Go to Transaction History 2. Filter by date range and type 3. Export as PDF	Date: 01/05/25 to 31/05/25 Type: Debit Click "Download PDF"	PDF report is generated and downloaded	As expected	Pass
Post Condition: User has filtered view and exported copy of transaction history.				

Test Case 6:

Project Name: Web-based Banking System			Test Designed by: Omi	
Test Case ID: WBS_6			Test Designed date: 3/06/25	
Test Priority: High			Test Executed by: Sayed Omi	
Module Name: Admin Dashboard			Test Execution date: 06/06/25	
Test Title: Admin Access and Account Management				
Description: Validate admin's ability to view and manage system users				
Precondition: Admin must be logged in with valid credentials				
Test Steps	Test Data	Expected Results	Actual Results	Status (Pass/Fail)
1. Log in as Admin 2. Navigate to Dashboard 3. View list of users 4. Deactivate a user account	Username: adminUser Password: Admin@123 Username: demoUser9	User status changes to "inactive"	As expected	Pass
Post Condition: Admin successfully manages user accounts and views system reports.				

8. ITEM PASS/FAIL CRITERIA

The testing team for the Web-Based iBanking System will consist of the following roles:

- Unit Test Manager
- Test Analyst
- Project Sponsor
- Developer

Team Responsibilities:

- Ensure code is error-free and compiles without issues.
- Test cases will be considered Passed only when the success rate is 100%.
- Any failed test or error encountered will be resolved promptly before proceeding.
- All executed tests will be properly documented with outcomes and observations.

Outcome Summary:

A total of 6 test cases were executed during system testing. Initially, 80% of the test cases passed successfully, while 20% failed due to query-related database issues. After resolving the database logic errors and reapplying the failed test cases, all test cases passed successfully, confirming system stability and correctness.

5. TEST DELIVERABLES

The following deliverables were prepared and maintained throughout the testing process to ensure traceability, transparency, and quality assurance:

- **Test Specification Document** – A detailed list of all scenarios and modules to be tested
- **Test Strategy** – High-level description of the overall testing approach and phases
- **Test Scenarios** – Descriptions of real-world actions to ensure full process flow coverage
- **Test Data** – Input data used to simulate various user and system interactions
- **Defect Reports** – Detailed tracking of bugs found, including severity and resolution status
- **Test Design** – Structured layout of test cases, including preconditions, steps, and expected outcomes
- **Test Status Report** – Regular updates on test execution progress and success rates
- **Summary Report** – Final evaluation of the overall testing results, including pass/fail percentages and observations

These deliverables collectively ensure that the iBanking system meets its functional, security, and usability goals before final deployment.

10. STAFFING AND TRAINING NEEDS

- Use certification programs to validate team members' skills and encourage them to stay updated with new testing methods and tools.
- Promote software testing and computing as part of continuous learning, even in fields such as applied physics and systems engineering.

➢ Provide interactive documentation through workbooks and internal wikis to help team members understand processes and reduce onboarding time.

➢ Utilize online platforms (e.g., video tutorials, LMS portals) to conduct training and share technical knowledge remotely.

➢ Emphasize industry-relevant skills that make team members more marketable for both academic and non-academic roles.

We require a dedicated and skilled team to successfully complete the "Web-Based Banking System" project. Well-trained professionals ensure that the project is executed efficiently, within time and budget, while maintaining high quality standards.

➢ Five full-time testers will be needed, each with a deep understanding of various levels of testing (unit, integration, system, UAT).

➢ Testers should be proficient in using tools such as Selenium, Postman, and SQL for automation and data validation.

➢ Familiarity with banking operations, especially functionalities involving ATM systems and transactional modules, will be considered essential.

➢ All testers will undergo specific training on the system's features, especially modules related to fund transfers, user authentication, and real-time processing.

By having the right people with the right training, the iBanking System project team can ensure robust quality assurance across all functional and non-functional requirements.

11. RESPONSIBILITIES

- The project team leader is responsible for verifying all test plans.
- The test lead is responsible for writing the test plan.
- The test managers are responsible for writing test cases.
- The entire project team will participate in the system review.
- The developer is responsible for fixing bugs; the tester is responsible for detecting bugs rather than fixing bugs.
- The administration is responsible for verifying the test results.

12. TESTING SCHEDULE

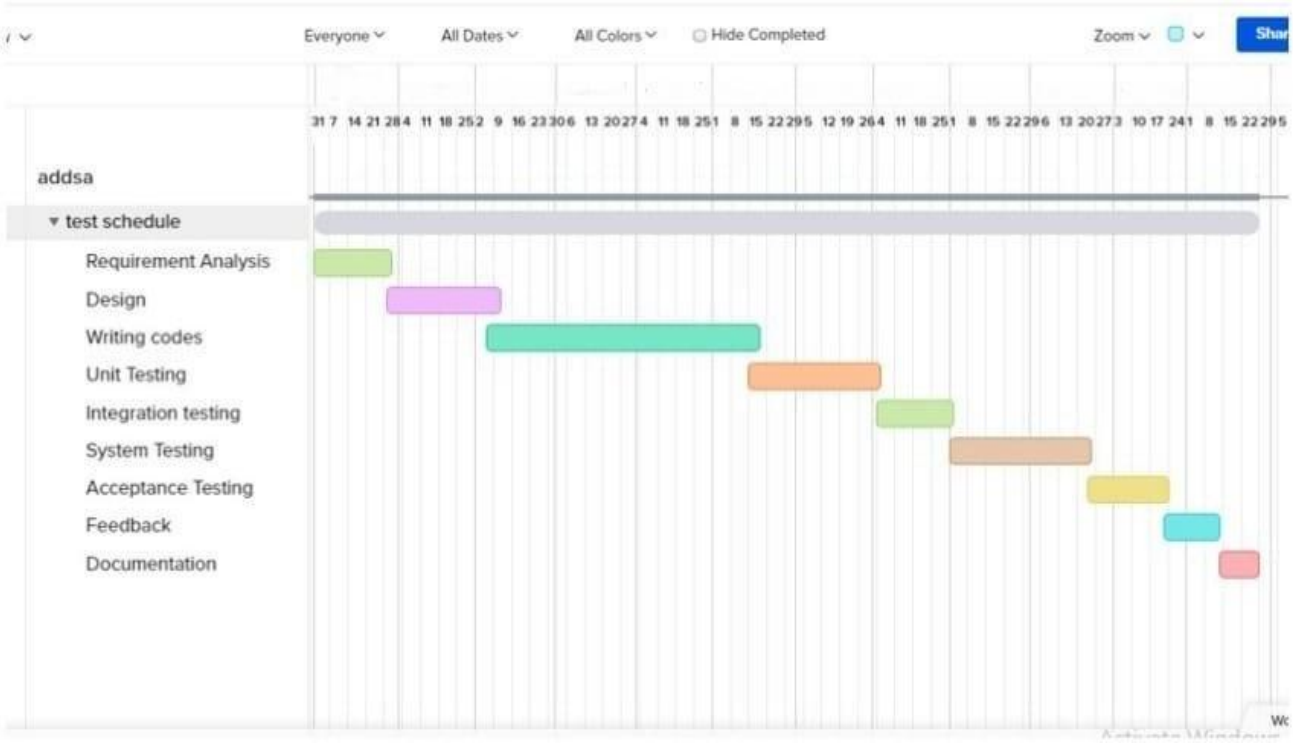


Fig: Testing Schedule

13. PLANNING RISKS AND CONTINGENCIES

1. **Limited Development Resources**

Risk: The development team might be understaffed due to one or more developer's being unavailable during the project timeline, which could delay feature development and bug fixing.

Contingency: If development resources become a bottleneck, project timelines for specific milestones will be adjusted. The project manager will prioritize critical features (e.g., transaction processing, user management) to ensure core functionality is delivered on time, while less critical features may be deferred to future iterations.

2. **Client Training and Transition Delays**

Risk: The transition from manual banking processes to the new system may take longer due to user resistance or lack of sufficient training, affecting system adoption.

Contingency: If training becomes an issue, additional training sessions will be scheduled, and a phased rollout approach will be considered, allowing users to gradually transition to the new system while continuing to use manual processes in parallel.

3. User Acceptance Testing (UAT) Delays

Risk: Admin, employee, and merchant users may be unavailable to perform UAT due to scheduling conflicts, which could delay the testing and feedback process.

Contingency: If end-users are unable to participate in UAT within the planned timeframe, alternative staff or stakeholders will be invited to conduct the testing. If no suitable alternatives are available, the UAT timeline will be extended to allow for proper testing without bypassing any processes.